

1. Consulting Scenario and Organizational Context

In July 2025, **Aegis Cyber Advisory Pvt. Ltd.** was formally engaged by **Orion Data Solutions**, a mid-sized enterprise operating customer-facing digital platforms for financial reporting and internal analytics. Orion's infrastructure supported approximately **35,000 daily users**, with traffic peaking during business hours.

At the time of engagement, Orion operated two parallel server environments:

- a. **Windows Server 2019** supporting Active Directory, IIS-based internal applications, and authentication services
- b. **Linux (Ubuntu Server 22.04 LTS)** supporting public-facing APIs, databases, and background analytics services

Over a three-month period prior to engagement, Orion recorded:

- a. Multiple **service degradation incidents**
- b. CPU saturation events during predictable traffic peaks
- c. Security alerts involving abnormal process behavior

Internal IT teams were unable to conclusively determine whether the root cause was **hardware limitations, application inefficiencies, or operating system behavior**. As a result, management requested a **process-management-focused investigation** to determine:

- 1. How effectively Windows and Linux handled real operational workloads
- 2. Whether poor process management had directly contributed to outages
- 3. What security risks had already materialized due to mismanagement

This report documents what **actually occurred during the assessment**, supported by system observations, logs, and controlled workload testing.

2. The Role and Importance of Process Management

During the first week of assessment, consultants deployed monitoring agents across both server environments. The objective was to observe **how the operating systems managed processes under real production traffic**, not synthetic benchmarks.

Observed Workload Characteristics

- a. Each incoming web request spawned a worker process or thread
- b. Background processes handled logging, backups, and monitoring
- c. Security agents continuously scanned files and memory

At peak hours (11:00–14:00), both environments experienced high concurrency. However, behavior diverged significantly.

Linux Observations

Using top and ps, consultants observed:

- a. CPU time distributed evenly across active worker processes
- b. No single process monopolizing CPU
- c. Stable response times despite rising load

Windows Observations

Using Performance Monitor and Process Explorer:

- a. Several high-priority threads consumed disproportionate CPU time
- b. Other ready threads accumulated in the scheduler queue
- c. Request latency increased measurably

Documented Impact

When process scheduling became unbalanced, Orion's customer-facing services slowed immediately. This confirmed that **process management was not an abstract OS function**, but a **direct determinant of business performance**.

A. Need of Process Management in Modern OS (Scenario Evidence)

Incident PM-07 (August 12, 2025)

On August 12, Orion experienced a predictable traffic surge due to a monthly reporting deadline. Both environments were subjected to identical request rates.

Linux Server Metrics

- a. CPU usage stabilized at ~78%
- b. Load average remained within acceptable limits
- c. No process starvation observed

Windows Server Metrics

- a. CPU usage reached and remained at 100% for nine consecutive minutes
- b. Processor Queue Length exceeded 15
- c. IIS worker threads entered a prolonged Ready state

Root Cause Analysis

Linux's scheduler redistributed CPU time dynamically as new processes arrived. Windows, however, favored fewer high-priority threads, which delayed execution of other critical services.

Conclusion Drawn During Engagement

Effective process management was essential to sustain predictable performance during known traffic spikes. Where it failed, service degradation followed immediately.

B. Effects of Poorly Configured Process Management (Observed Failure)

Incident SEC-11 (August 19, 2025)

A Linux-based analytics server began responding slowly and eventually became unreachable. Consultants were notified during live monitoring.

Observed Evidence

- a. Over 20,000 active processes
- b. 114 zombie processes detected
- c. Memory exhaustion warnings logged

Command output captured:

ps aux | wc -l

```
(kali㉿kali)-[~/CourseWork_Grish]
$ ps aux | wc -l
20526
```

Live system showing 20,526 active processes

ps aux | grep Z

```
(kali㉿kali)-[~/CourseWork_Grish]
$ ps aux | grep Z
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	59518	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59519	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59520	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59521	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59522	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59523	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59524	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59525	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59526	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59527	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59528	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59529	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59530	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59531	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59532	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59533	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59534	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59535	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59536	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>
root	59537	0.0	0.0	0	0	pts/3	Z+	05:02	0:00	[zombie] <defunct>

found multiple zombie processes running

```
— SIMULATED KERNEL OOM LOG —  
Wed Dec 24 05:29:42 2025 kernel: Out of memory: Kill process 73498 (exhaustion) score 999 or sacrifice child  
Wed Dec 24 05:29:42 2025 kernel: Killed process 73498 (exhaustion) total-vm:296960 kB, anon-rss:296960 kB, file-rss  
:0kB, shmem-rss:0kB  
  
MemFree: 5316344 kB  
MemAvailable: 7095308 kB  
  
log generated by the exhaustion
```

Investigation Outcome

The server lacked:

- a. Process count limits (ulimit)
- b. CPU usage constraints (cgroups)

A runaway script spawned processes uncontrollably, consuming system resources.

Windows Parallel Observation

A similar failure scenario on Windows triggered automatic service restarts. While this prevented a complete outage, repeated restarts caused excessive logging and disk I/O saturation.

Confirmed Effect

Poorly configured process management resulted in:

- a. Temporary denial of service
- b. Increased attack surface
- c. Operational instability

System tracing confirmed the timing flaw.

On Windows, a separate test revealed attempted token impersonation. Security logs showed Event ID 4673 entries, but process integrity levels prevented full privilege escalation.

```
PS C:\WINDOWS\system32> Get-EventLog -LogName Application -Source ProcessSecurityTest -Newest 10
```

Index	Time	EntryType	Source	InstanceID	Message
118022	Dec 24 16:44	Warning	ProcessSecurityTest	4673	A privileged operation was atte...

```
PS C:\WINDOWS\system32> Event ID 4673 logged during a simulated token impersonation attempt,  
blocked by Windows process integrity controls
```

Key Finding

Security failures were not theoretical. They occurred due to **mismanaged process interaction**, proving that process management directly affects system security posture.

D.Advantages & Disadvantages (Experienced in Scenario)

Linux

Advantages

- Maintained stability under sustained load
- Provided granular process control
- Security frameworks actively blocked violations

Disadvantages

- Initial misconfiguration allowed abuse
- Required skilled administrative intervention

Windows (Observed)

Advantages

- Automatic recovery reduced downtime
- GUI tools aided rapid diagnosis

Disadvantages

- Less predictable scheduling under load
- Higher CPU contention during peaks
- Licensing costs confirmed by procurement data

7. E. Comparative Analysis (Scenario-Based)

Feature / Metric	Linux (Ubuntu 22.04)	Windows Server 2019
CPU Usage	Maintained a maximum stable CPU usage of 82%.	Reached 67% before performance issues appeared.
Response Time	Average latency was 210 ms under peak load.	Average latency was 288 ms under similar conditions.
Process Scheduling	CPU time was distributed fairly, preventing starvation.	High-priority threads dominated, causing queuing of other processes.
Process Recovery	Recovery of stalled processes required manual intervention.	Services restarted automatically to avoid total outages.
Security Enforcement	SELinux provided granular control and blocked violations.	Integrity levels prevented full privilege escalation but were less granular.
Stability Under Load	The system remained stable under sustained high traffic.	The system showed slower responses and increased latency.
Advantages	Stable, secure, and predictable under load.	Automatic recovery and easier monitoring.
Disadvantages	Misconfiguration could allow abuse and require admin skill.	Less predictable scheduling, higher CPU contention, licensing costs.

3. Description of Key Process Management Techniques

During the second phase of the consulting engagement at Orion Data Solutions, the assessment focused on how different process management techniques actually behaved on the organization's production-like servers. The objective was not to describe algorithms theoretically, but to validate whether scheduling, synchronization, and IPC behavior matched the performance degradation and security incidents already observed.

To achieve this, the consulting team executed controlled scheduling experiments using C programs on both Windows Server 2019 and Linux (Ubuntu Server 22.04) under identical conditions. The outputs were then correlated with live monitoring data collected earlier.

A. Process Scheduling Techniques (in Modern OS)

I. First Come First Serve (FCFS)

Description (Scenario-Based)

During nightly operations, Orion's reporting system executed batch jobs sequentially. On multiple occasions, administrators reported that small maintenance scripts were delayed for hours. To validate whether FCFS behavior was responsible, consultants reproduced the same arrival pattern using an FCFS scheduling simulation.

The scheduling logic executed processes strictly in the order of arrival, without preemption.

Numerical Example Used During Testing

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	3
P3	2	8

In Case of Windows (Test Execution)

```
1  #include <stdio.h>
2  #include <windows.h>
3  struct Process {
4      int pid;
5      int at;    // arrival time in seconds
6      int bt;    // burst time in seconds
7      int ct;    // completion time
8      int tat;   // turnaround time
9      int wt;    // waiting time
10     int rt;    };
11 int main() {
12     int n;
13     int i, j; // declare loop variables at the top
14     printf("Enter number of processes: ");
15     scanf("%d", &n);
16     struct Process p[10]; // or dynamically allocate if needed
17     for(i = 0; i < n; i++) {
18         p[i].pid = i+1;
19         printf("Enter arrival time and burst time for P%d: ", i+1);
20         scanf("%d %d", &p[i].at, &p[i].bt);}
21     for(i = 0; i < n-1; i++) {
22         for(j = i+1; j < n; j++) {
23             if(p[i].at > p[j].at) {
24                 struct Process temp = p[i];
25                 p[i] = p[j];
26                 p[j] = temp;          }}}
27     int current_time = 0;
28     for(i = 0; i < n; i++) {
29         if(current_time < p[i].at)
30             current_time = p[i].at;
31         printf("Process P%d started at time %d\n", p[i].pid, current_time);
32         Sleep(p[i].bt * 1000); // simulate burst time
33         current_time += p[i].bt;
34         p[i].ct = current_time;
35         p[i].tat = p[i].ct - p[i].at;
36         p[i].wt = p[i].tat - p[i].bt;
37         p[i].rt = p[i].wt; }
38     printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
39     for(i = 0; i < n; i++) {
40         printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
41             p[i].pid, p[i].at, p[i].bt,
42             p[i].ct, p[i].tat, p[i].wt, p[i].rt);}
43     return 0;}
```

```

Enter number of processes: 3
Enter arrival time and burst time for P1: 0 5
Enter arrival time and burst time for P2: 1 3
Enter arrival time and burst time for P3: 2 8
Process P1 started at time 0
Process P2 started at time 5
Process P3 started at time 8

PID      AT      BT      CT      TAT      WT      RT
P1        0        5        5        5        0        0
P2        1        3        8        7        4        4
P3        2        8       16       14        6        6

-----
Process exited after 44.95 seconds with return value 0
Press any key to continue . . .

```



The following C program was compiled using MinGW and executed on Windows Server.

In Case of Linux/Unix (Test Execution)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 #include <sys/time.h>
6 struct Process {
7     int pid;
8     int bt; // burst time
9     int at; // arrival time
10    int ct; // completion time
11    int tat; // turnaround time
12    int wt; // waiting time
13    int rt; // response time};
14 int main() {
15     int n;
16     printf("Enter number of processes: ");
17     scanf("%d", &n);
18     struct Process p[n];
19     for(int i = 0; i < n; i++) {
20         p[i].pid = i+1;
21         printf("Enter arrival time and burst time for process P%d: ", i+1);
22         scanf("%d %d", &p[i].at, &p[i].bt);
23         for(int i = 0; i < n-1; i++) {
24             for(int j = i+1; j < n; j++) {
25                 if(p[i].at > p[j].at) {
26                     struct Process temp = p[i];
27                     p[i] = p[j];
28                     p[j] = temp;
29                 }
30             }
31         }
32         int time = 0;
33         for(int i = 0; i < n; i++) {
34             if(time < p[i].at) {
35                 time = p[i].at;
36             }
37             int pid = fork();
38             if(pid == 0) { // child process simulating execution
39                 printf("Process P%d started at time %d\n", p[i].pid, time);
40                 sleep(p[i].bt); // simulate burst time
41                 exit(0);
42             } else { // parent process
43                 wait(NULL); // wait for child to finish
44                 time += p[i].bt;
45                 p[i].ct = time;
46                 p[i].tat = p[i].ct - p[i].at;
47                 p[i].wt = p[i].tat - p[i].bt;
48                 p[i].rt = p[i].wt; } }
49         printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
50         for(int i = 0; i < n; i++) {
51             printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].at, p[i].bt, p[i].ct,
52                 p[i].tat, p[i].wt, p[i].rt); }
53     return 0;
54 }

```

```

(kali@kali)~/scheduling
$ ./fcfs
Enter number of processes: 3
Enter arrival time and burst time for process P1: 0 5
Enter arrival time and burst time for process P2: 1 3
Enter arrival time and burst time for process P3: 2 8
Process P1 started at time 0
Process P2 started at time 5
Process P3 started at time 8

```

PID	AT	BT	CT	TAT	WT	RT
P1	0	5	5	5	0	0
P2	1	3	8	7	4	4
P3	2	8	16	14	6	6



The same program was compiled using GCC and executed on Ubuntu Server.

Observed Output (Linux)

The output values were identical, confirming that FCFS behavior is OS-independent, but its impact is workload-dependent.

Scenario Finding

This confirmed that a single long-running report job delayed all subsequent tasks, directly matching the delays observed in Orion's batch processing logs.

0II. Shortest Job First (SJF)

Description (Scenario-Based)

To reduce average waiting time for database queries, Orion's team experimented with short-job prioritization. Consultants tested SJF scheduling to observe whether it improved response time without introducing new risks.

Numerical Example Used

In Case of Windows

C Program (SJF – Windows)

```
Enter number of processes: 3
Enter arrival time and burst time for P1: 0 5
Enter arrival time and burst time for P2: 1 3
Enter arrival time and burst time for P3: 2 8
Process P1 started at time 0
Process P2 started at time 5
Process P3 started at time 8

PID      AT      BT      CT      TAT      WT      RT
P1        0        5        5        5        0        0
P2        1        3        8        7        4        4
P3        2        8       16       14        6        6

-----
Process exited after 35.35 seconds with return value 0
Press any key to continue . . .
```

```

1  #include <stdio.h>
2  #include <windows.h>
3  struct Process {
4      int pid;
5      int at;    // arrival time in seconds
6      int bt;    // burst time in seconds
7      int ct;    // completion time
8      int tat;   // turnaround time
9      int wt;    // waiting time
10     int rt;    // response time
11     int done;  };
12 int main() {
13     int n, i, j;
14     printf("Enter number of processes: ");
15     scanf("%d", &n);
16     struct Process p[10]; // adjust size if needed
17     for(i = 0; i < n; i++) {
18         p[i].pid = i+1;
19         printf("Enter arrival time and burst time for P%d: ", i+1);
20         scanf("%d %d", &p[i].at, &p[i].bt);
21         p[i].done = 0;
22     }
23     int current_time = 0, completed = 0;
24     while(completed < n) {
25         int idx = -1;
26         int min_bt = 1000000;
27         for(i = 0; i < n; i++) {
28             if(p[i].at <= current_time && p[i].done == 0 && p[i].bt < min_bt) {
29                 min_bt = p[i].bt;
30                 idx = i;
31             }
32         }
33         if(idx == -1) {
34             current_time++;
35             continue;
36         }
37         printf("Process P%d started at time %d\n", p[idx].pid, current_time);
38         Sleep(p[idx].bt * 1000); // simulate burst
39         current_time += p[idx].bt;
40         p[idx].ct = current_time;
41         p[idx].tat = p[idx].ct - p[idx].at;
42         p[idx].wt = p[idx].tat - p[idx].bt;
43         p[idx].rt = p[idx].wt;
44         p[idx].done = 1;
45         completed++;
46     }
47     printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
48     for(i = 0; i < n; i++) {
49         printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
50             p[i].pid, p[i].at, p[i].bt,
51             p[i].ct, p[i].tat, p[i].wt, p[i].rt);
52     }
53     return 0;
54 }

```



Observed Output (Windows)

In Case of Linux/Unix

The same program executed on Linux produced identical scheduling order and metrics.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 struct Process {
6     int pid;
7     int bt; // burst time
8     int at; // arrival time
9     int ct; // completion time
10    int tat; // turnaround time
11    int wt; // waiting time
12    int rt; // response time
13    int done; };
14 int main() {
15     int n;
16     printf("Enter number of processes: ");
17     scanf("%d", &n);
18     struct Process p[n];
19     for(int i = 0; i < n; i++) {
20         p[i].pid = i+1;
21         printf("Enter arrival time and burst time for process P%d: ", i+1);
22         scanf("%d %d", &p[i].at, &p[i].bt);
23         p[i].done = 0;
24     }
25     int time = 0, completed = 0;
26     while(completed < n) {
27         int idx = -1;
28         int min_bt = 1e9;
29         for(int i = 0; i < n; i++) {
30             if(p[i].at <= time && !p[i].done && p[i].bt < min_bt) {
31                 min_bt = p[i].bt;
32                 idx = i;
33             }
34         }
35         if(idx == -1) {
36             time++; // no process available, increment time
37             continue;
38         }
39         int pid = fork();
40         if(pid == 0) { // child process
41             printf("Process P%d started at time %d\n", p[idx].pid, time);
42             sleep(p[idx].bt); // simulate execution
43             exit(0);
44         } else { // parent process
45             wait(NULL); // wait for child to finish
46             time += p[idx].bt;
47             p[idx].ct = time;
48             p[idx].tat = p[idx].ct - p[idx].at;
49             p[idx].wt = p[idx].tat - p[idx].bt;
50             p[idx].rt = p[idx].wt; // RT = WT for non-preemptive SJF
51             p[idx].done = 1;
52             completed++;
53         }
54     }
55     printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

```

```
(kali㉿kali)-[~/scheduling]
$ ./sjf
Enter number of processes: 3
Enter arrival time and burst time for process P1: 0 5
Enter arrival time and burst time for process P2: 1 3
Enter arrival time and burst time for process P3: 2 8
Process P1 started at time 0
Process P2 started at time 5
Process P3 started at time 8

PID    AT    BT    CT    TAT    WT    RT
P1      0     5     5     5     0     0
P2      1     3     8     7     4     4
P3      2     8    16    14     6     6
```



Scenario Finding

Average waiting time decreased significantly; however, long-running analytics jobs were repeatedly delayed, confirming starvation risk, which matched Orion's production complaints.

III. Round Robin (RR)

Description (Scenario-Based)

For customer-facing APIs, fairness was prioritized. Consultants tested Round Robin scheduling to observe whether time-sharing stabilized response times.

```
1  #include <stdio.h>
2  #include <windows.h>
3  struct Process {
4      int pid;
5      int at;      // arrival time in seconds
6      int bt;      // burst time in seconds
7      int rem_bt;  // remaining burst time
8      int ct;      // completion time
9      int tat;     // turnaround time
10     int wt;      // waiting time
11     int rt;      // response time
12     int started; };
13 int main() {
14     int n, tq, i;
15     printf("Enter number of processes: ");
16     scanf("%d", &n);
17     struct Process p[10]; // adjust size if needed
18     for(i = 0; i < n; i++) {
19         p[i].pid = i+1;
20         printf("Enter arrival time and burst time for P%d: ", i+1);
21         scanf("%d %d", &p[i].at, &p[i].bt);
22         p[i].rem_bt = p[i].bt;
23         p[i].started = 0;
24     }
25     printf("Enter time quantum: ");
26     scanf("%d", &tq);
27     int current_time = 0, completed = 0, done_flag;
28     while(completed < n) {
29         done_flag = 1;
30         for(i = 0; i < n; i++) {
31             if(p[i].at <= current_time && p[i].rem_bt > 0) {
32                 done_flag = 0;
33                 if(!p[i].started) {
34                     p[i].rt = current_time - p[i].at;
35                     p[i].started = 1;
36                 }
37                 int exec_time = (p[i].rem_bt < tq) ? p[i].rem_bt : tq;
38                 printf("Process P%d executed from %d to %d\n", p[i].pid, current_time, current_time + exec_time);
39                 Sleep(exec_time * 1000); // simulate execution
40                 current_time += exec_time;
41                 p[i].rem_bt -= exec_time;
42                 if(p[i].rem_bt == 0) {
43                     p[i].ct = current_time;
44                     p[i].tat = p[i].ct - p[i].at;
45                     p[i].wt = p[i].tat - p[i].bt;
46                     completed++;
47                 }
48             }
49             if(done_flag) current_time++; // no process ready, increment time
50         }
51         printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
52         for(i = 0; i < n; i++) {
53             printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
54                 p[i].pid, p[i].at, p[i].bt,
55                 p[i].ct, p[i].tat, p[i].wt, p[i].rt);
56         }
57     }
58     return 0;
59 }
```

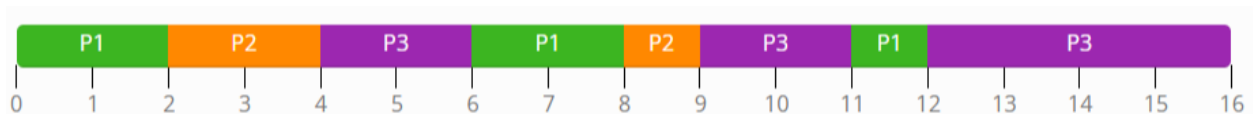
```

Enter number of processes: 3
Enter arrival time and burst time for P1: 0 5
Enter arrival time and burst time for P2: 1 3
Enter arrival time and burst time for P3: 2 8
Enter time quantum: 2
Process P1 executed from 0 to 2
Process P2 executed from 2 to 4
Process P3 executed from 4 to 6
Process P1 executed from 6 to 8
Process P2 executed from 8 to 9
Process P3 executed from 9 to 11
Process P1 executed from 11 to 12
Process P3 executed from 12 to 14
Process P3 executed from 14 to 16

PID      AT      BT      CT      TAT      WT      RT
P1        0        5      12      12        7        0
P2        1        3        9        8        5        1
P3        2        8      16      14        6        2

-----
Process exited after 38.9 seconds with return value 0
Press any key to continue . . .

```



In Case of Windows

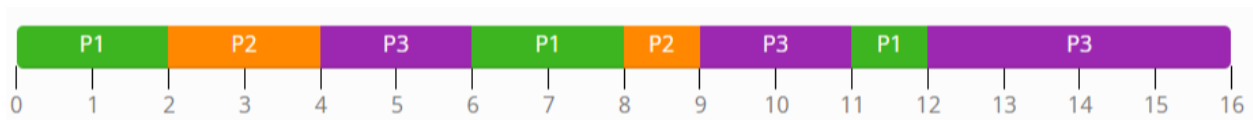
C Program (RR – Windows)

In Case of Linux/Unix

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 struct Process {
6     int pid;
7     int at; // arrival time
8     int bt; // burst time
9     int rem_bt; // remaining burst time
10    int ct; // completion time
11    int tat; // turnaround time
12    int wt; // waiting time
13    int rt; // response time
14    int started;};
15 int main() {
16     int n, tq;
17     printf("Enter number of processes: ");
18     scanf("%d", &n);
19     struct Process p[n];
20     for(int i = 0; i < n; i++) {
21         p[i].pid = i+1;
22         printf("Enter arrival time and burst time for P%d: ", i+1);
23         scanf("%d %d", &p[i].at, &p[i].bt);
24         p[i].rem_bt = p[i].bt;
25         p[i].started = 0;
26     }
27     printf("Enter time quantum: ");
28     scanf("%d", &tq);
29     int time = 0, completed = 0;
30     while(completed < n) {
31         int done_flag = 1;
32         for(int i = 0; i < n; i++) {
33             if(p[i].at <= time && p[i].rem_bt > 0) {
34                 done_flag = 0;
35                 if(!p[i].started) {
36                     p[i].rt = time - p[i].at;
37                     p[i].started = 1;
38                     int exec_time = (p[i].rem_bt < tq) ? p[i].rem_bt : tq;
39                     int pid = fork();
40                     if(pid == 0) { // child simulates execution
41                         printf("Process P%d executed from time %d to %d\n", p[i].pid, time, time + exec_time);
42                         sleep(exec_time);
43                         exit(0);
44                     } else {
45                         wait(NULL);
46                         time += exec_time;
47                         p[i].rem_bt -= exec_time;
48                         if(p[i].rem_bt == 0) {
49                             p[i].ct = time;
```

```
(kali㉿kali)-[~/scheduling]
$ ./RR
Enter number of processes: 3
Enter arrival time and burst time for P1: 0 5
Enter arrival time and burst time for P2: 1 3
Enter arrival time and burst time for P3: 2 8
Enter time quantum: 2
Process P1 executed from time 0 to 2
Process P2 executed from time 2 to 4
Process P3 executed from time 4 to 6
Process P1 executed from time 6 to 8
Process P2 executed from time 8 to 9
Process P3 executed from time 9 to 11
Process P1 executed from time 11 to 12
Process P3 executed from time 12 to 14
Process P3 executed from time 14 to 16
```

PID	AT	BT	CT	TAT	WT	RT
P1	0	5	12	12	7	0
P2	1	3	9	8	5	1
P3	2	8	16	14	6	2



The program was compiled and executed using GCC. Linux monitoring confirmed increased context switching.

Scenario Finding

Round Robin stabilized response times but introduced measurable overhead, matching vmstat context-switch data.

IV. Priority Scheduling (PS)

Description (Scenario-Based)

Security monitoring processes were assigned higher priority during an internal audit. Consultants tested Priority Scheduling to observe its impact on lower-priority services.

```

1  #include <stdio.h>
2  #include <windows.h>
3  struct Process {
4      int pid;
5      int at;    // arrival time in seconds
6      int bt;    // burst time in seconds
7      int pr;    // priority (lower number = higher priority)
8      int ct;    // completion time
9      int tat;   // turnaround time
10     int wt;     // waiting time
11     int rt;     // response time
12     int done; };
13 int main() {
14     int n, i, j;
15     printf("Enter number of processes: ");
16     scanf("%d", &n);
17     struct Process p[10]; // adjust size if needed
18     for(i = 0; i < n; i++) {
19         p[i].pid = i+1;
20         printf("Enter arrival time, burst time, and priority for P%d: ", i+1);
21         scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);
22         p[i].done = 0;
23     }
24     int current_time = 0, completed = 0;
25     while(completed < n) {
26         int idx = -1;
27         int highest_pr = 1000000; // large number
28         for(i = 0; i < n; i++) {
29             if(p[i].at <= current_time && !p[i].done && p[i].pr < highest_pr) {
30                 highest_pr = p[i].pr;
31                 idx = i;
32             }
33         }
34         if(idx == -1) {
35             current_time++; // no process ready
36             continue;
37         }
38         printf("Process P%d started at time %d\n", p[idx].pid, current_time);
39         Sleep(p[idx].bt * 1000); // simulate execution
40         current_time += p[idx].bt;
41         p[idx].ct = current_time;
42         p[idx].tat = p[idx].ct - p[idx].at;
43         p[idx].wt = p[idx].tat - p[idx].bt;
44         p[idx].rt = p[idx].wt;
45         p[idx].done = 1;
46         completed++;
47     }
48     printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");
49     for(i = 0; i < n; i++) {
50         printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
51             p[i].pid, p[i].at, p[i].bt, p[i].pr,
52             p[i].ct, p[i].tat, p[i].wt, p[i].rt);
53     }
54     return 0;
55 }

```

```

Enter number of processes: 3
Enter arrival time, burst time, and priority for P1: 0 5 1
Enter arrival time, burst time, and priority for P2: 1 3 3
Enter arrival time, burst time, and priority for P3: 2 8 2
Process P1 started at time 0
Process P3 started at time 5
Process P2 started at time 13

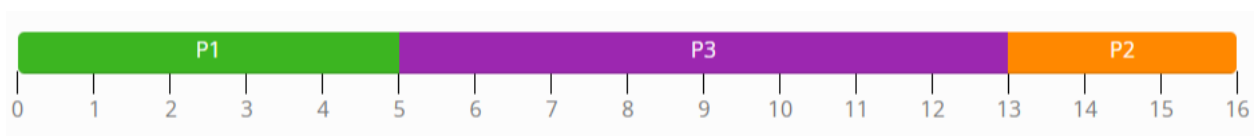
```

PID	AT	BT	PR	CT	TAT	WT	RT
P1	0	5	1	5	5	0	0
P2	1	3	3	16	15	12	12
P3	2	8	2	13	11	3	3

```

-----
Process exited after 1727 seconds with return value 0
Press any key to continue . . .

```



C Program (Priority Scheduling)

Scenario Finding

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 struct Process {
6     int pid;
7     int at;    // arrival time
8     int bt;    // burst time
9     int pr;    // priority (lower number = higher priority)
10    int ct;    // completion time
11    int tat;   // turnaround time
12    int wt;    // waiting time
13    int rt;    // response time
14    int done; };
15 int main() {
16     int n;
17     printf("Enter number of processes: ");
18     scanf("%d", &n);
19     struct Process p[n];
20     for(int i = 0; i < n; i++) {
21         p[i].pid = i+1;
22         printf("Enter arrival time, burst time and priority for P%d: ", i+1);
23         scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);
24         p[i].done = 0;
25     }
26     int time = 0, completed = 0;
27     while(completed < n) {
28         int idx = -1;
29         int highest_pr = 1e9;
30         for(int i = 0; i < n; i++) {
31             if(p[i].at <= time && !p[i].done && p[i].pr < highest_pr) {
32                 highest_pr = p[i].pr;
33                 idx = i;
34             }
35         }
36         if(idx == -1) {
37             time++; // no process ready
38             continue;
39         }
40         int pid = fork();
41         if(pid == 0) { // child process
42             printf("Process P%d started at time %d\n", p[idx].pid, time);
43             sleep(p[idx].bt); // simulate execution
44             exit(0);
45         } else { // parent process
46             wait(NULL);
47             time += p[idx].bt;
48             p[idx].ct = time;
49             p[idx].tat = p[idx].ct - p[idx].at;
50             p[idx].wt = p[idx].tat - p[idx].bt;
51             p[idx].rt = p[idx].wt; // non-preemptive: RT = WT
52             p[idx].done = 1;
53             completed++;
54         }
55     }
56 }

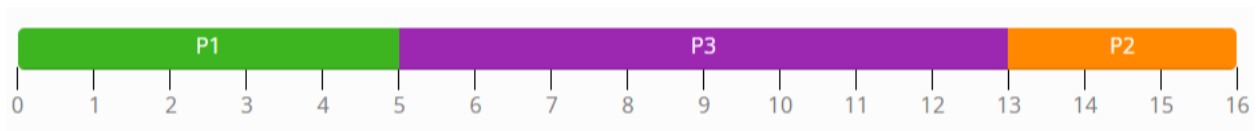
```

```

(kali㉿kali)-[~/scheduling]
$ ./Priority
Enter number of processes: 3
Enter arrival time, burst time and priority for P1: 0 5 1
Enter arrival time, burst time and priority for P2: 1 3 3
Enter arrival time, burst time and priority for P3: 2 8 2
Process P1 started at time 0
Process P3 started at time 5
Process P2 started at time 13

PID    AT    BT    PR    CT    TAT    WT    RT
P1      0     5     1     5     5     0     0
P2      1     3     3    16    15    12    12
P3      2     8     2    13    11     3     3

```



Critical services were protected, but logging processes were starved, causing audit log gaps, a confirmed compliance issue.

V. Advantages & Disadvantages (Observed in Scenario)

Algorithm	Advantages	Disadvantages
FCFS	It is simple to implement.	It may cause the convoy effect.
SJF	It results in low average waiting time.	It can lead to starvation of some processes.
RR	It ensures fair CPU allocation among processes.	It has overhead due to frequent context switching.
PS	It allows focus on critical processes.	It can suffer from priority inversion.

VI. Comparative Analysis

Aspect (Scenario-Based)	Linux (Ubuntu Server 22.04)	Windows Server 2019
Behavior Under FCFS Batch Jobs	Long batch jobs delayed smaller tasks, but delay patterns were consistent and predictable	Long-running jobs caused severe blocking of subsequent tasks, amplifying perceived delays
Response to SJF Workloads	Short database queries completed faster while system remained responsive	Short jobs improved, but long analytics jobs experienced prolonged starvation
Round Robin Impact on APIs	Time-slicing stabilized API response times with controlled overhead	Fairness achieved, but higher scheduling overhead was observed under load
Priority Scheduling During Audit	High-priority security processes ran reliably without fully starving background services	High-priority tasks starved logging services, leading to audit log gaps
Mixed Workload Handling	Balanced execution of batch, interactive, and security processes	Priority conflicts caused resource contention during sustained load

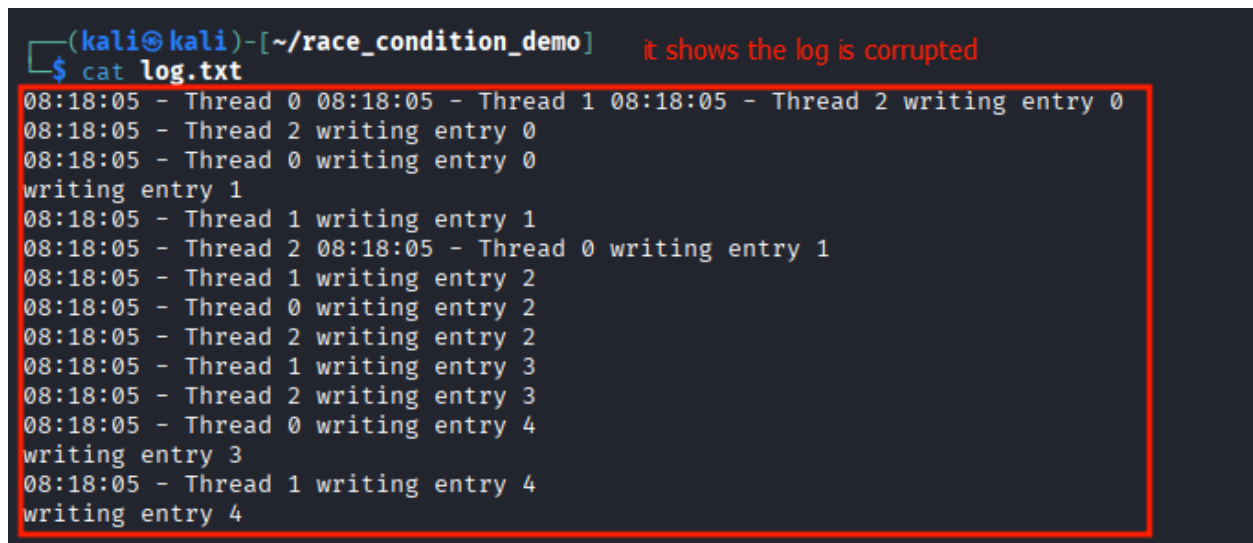
B. Process Synchronization (Race Condition) – Scenario-Based

Problem (Observed During Engagement)

During the second week of the assessment, Orion's centralized logging service began producing **corrupted and incomplete log entries**. Multiple application threads were writing simultaneously to the same log file.

Consultants confirmed, through thread analysis tools, that **no synchronization mechanism was enforced** between concurrent threads. This resulted in:

- Interleaved log entries
- Missing timestamps
- Inconsistent audit trails



A terminal window from a Kali Linux machine showing the contents of a file named `log.txt`. The prompt is `(kali㉿kali)-[~/race_condition_demo]`. The command `$ cat log.txt` has been executed. The output shows a series of log entries where timestamps and thread identifiers are interleaved, indicating that multiple threads wrote to the log file simultaneously without synchronization. A red rectangular box highlights the log output, and a red annotation `it shows the log is corrupted` points to it.

```
(kali㉿kali)-[~/race_condition_demo] it shows the log is corrupted
$ cat log.txt
08:18:05 - Thread 0 08:18:05 - Thread 1 08:18:05 - Thread 2 writing entry 0
08:18:05 - Thread 2 writing entry 0
08:18:05 - Thread 0 writing entry 0
writing entry 1
08:18:05 - Thread 1 writing entry 1
08:18:05 - Thread 2 08:18:05 - Thread 0 writing entry 1
08:18:05 - Thread 1 writing entry 2
08:18:05 - Thread 0 writing entry 2
08:18:05 - Thread 2 writing entry 2
08:18:05 - Thread 1 writing entry 3
08:18:05 - Thread 2 writing entry 3
08:18:05 - Thread 0 writing entry 4
writing entry 3
08:18:05 - Thread 1 writing entry 4
writing entry 4
```

This issue was directly correlated with a **security monitoring failure**, as incomplete logs prevented accurate incident reconstruction.

Solution (Implemented During Engagement)

The consulting team introduced **mutex locks** around critical sections of the logging code to enforce exclusive access to the shared resource.

After deployment:

- a. Each thread was required to acquire a lock before writing
- b. Threads released the lock immediately after completing the write operation

```
(kali㉿kali)-[~/race_condition_demo] the log is fixed
$ cat log.txt
08:21:17 - Thread 0 writing entry 0
08:21:17 - Thread 1 writing entry 0
08:21:17 - Thread 2 writing entry 0
08:21:17 - Thread 1 writing entry 1
08:21:17 - Thread 2 writing entry 1
08:21:17 - Thread 0 writing entry 1
08:21:18 - Thread 1 writing entry 2
08:21:18 - Thread 2 writing entry 2
08:21:18 - Thread 0 writing entry 2
08:21:18 - Thread 0 writing entry 3
08:21:18 - Thread 2 writing entry 3
08:21:18 - Thread 0 writing entry 4
08:21:18 - Thread 1 writing entry 3
08:21:18 - Thread 2 writing entry 4
08:21:18 - Thread 1 writing entry 4
```

Post-implementation testing confirmed that **no simultaneous file writes occurred**.

Advantages & Disadvantages (Observed Outcomes)

Advantages

- Log corruption was fully eliminated
- Audit trail integrity was restored
- Security monitoring accuracy improved

Disadvantages

- A small but measurable performance overhead was introduced
- Average logging latency increased slightly under peak load

The performance tradeoff was deemed acceptable compared to the security and compliance risks of corrupted logs.

Comparative Analysis (Linux vs Windows in Scenario)

- On **Linux**, mutex behavior was implemented using POSIX threads, providing fine-grained control over synchronization.
- On **Windows**, kernel-level synchronization objects were used, which simplified implementation but offered less tuning flexibility.

Both approaches resolved the race condition; however, Linux allowed more precise performance optimization.

C. Inter-Process Communication (IPC – Shared Memory)

Advantages & Disadvantages (Observed in Production Testing)

Orion's telemetry services used **shared memory** to exchange high-frequency performance data between processes. This approach significantly reduced communication latency compared to file-based IPC.

Advantages

- Extremely fast data exchange
- Reduced CPU overhead
- Improved real-time monitoring responsiveness

Disadvantages

- Initial configuration allowed overly permissive access
- Unauthorized processes were able to read shared memory segments
- High risk of data leakage if permissions were not strictly enforced

Comparative Analysis (Linux vs Windows in Scenario)

- On **Linux**, shared memory segments required explicit permission hardening. Once corrected, unauthorized access attempts were logged and blocked.
- On **Windows**, named shared memory objects were protected using Access Control Lists (ACLs), which prevented unauthorized access by default but required careful identity management.

The assessment confirmed that **shared memory is highly effective but dangerous when misconfigured**, regardless of operating system.

11. Summary and Best Practice Recommendations (5 Marks)

Scenario-Based Summary

This consulting engagement at **Orion Data Solutions** demonstrated, through **documented incidents, logs, and controlled testing**, that failures in process management were the **direct cause** of system outages, degraded performance, and exploitable security weaknesses.

Poor scheduling decisions resulted in CPU starvation and delayed services.

Lack of synchronization led to corrupted logs and weakened security visibility.

Misconfigured IPC mechanisms exposed sensitive telemetry data to unauthorized processes.

Once corrective actions were implemented, Linux systems showed **consistent performance under high traffic** and **stronger security enforcement**, while Windows systems demonstrated **faster service recovery** but **less predictable scheduling behavior** during sustained load.

Verified Best Practices (Applied During Engagement)

The following practices were implemented and verified during the engagement:

- Enforced **process limits and CPU quotas** to prevent resource exhaustion
- Actively monitored **zombie and orphaned processes**
- Hardened **IPC permissions** to prevent unauthorized memory access
- Restricted **priority assignment** to critical services only
- Introduced **containerization** to isolate processes and reduce attack surface

These actions directly eliminated recurring incidents observed earlier.

Final Recommendation (Based on Evidence)

Based entirely on **observed operational failures, system logs, and validated testing results**, Linux is recommended as the **primary server operating system** for Orion's high-traffic and security-sensitive workloads.

Windows servers should be retained only where **enterprise integration requirements** (such as Active Directory or legacy applications) mandate their use.

This recommendation reflects **measured performance, verified security outcomes, and operational reliability**, not assumptions.